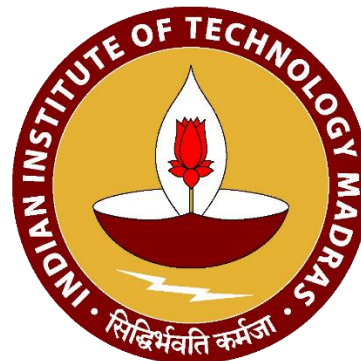


Python for Data Science

Web M. Tech Course DA5301W

Module 8: Visualisation in Python

Dr. P.S Jayadev



Contents

- Intro to Data Visualisation in Python
- Types of Variables
- Types of Plots – Based on Purpose
- Python Plotting Libraries – matplotlib, seaborn, plotly

Intro to Data Visualization in Python

- Why visualize data?
 - See patterns and trends that numbers alone don't reveal
 - Understand data shape & structure (spread, clusters, centrality)
 - Detect outliers
 - Understand relationship between variables
 - Simplify complex data into clear insights and support better decisions
 - Communicate results in an intuitive way
- Key Focus in this Module:
 - Different types of plots and when to use what
 - Visualization libraries: Matplotlib, seaborn, plotly – strengths and use cases

Types of Variables

1. **Numerical Variables:** Variables which can be measured and placed in ascending or descending order
 - a. **Continuous variables:** Numerical variables which can take continuous values
Eg. Person's height, weight, etc.
 - b. **Discrete variables:** Numerical variables which can take discretised values as such binary numbers, integers, whole numbers or any other discretisation
Eg. Pages in a book, count of objects
2. **Categorical Variables:** Variables that can be sorted into categories or groups
 - a. **Ordinal variables:** They can be ordered or ranked according to a scale but not measured
Eg. levels of temp (cold, warm, hot)
 - b. **Nominal variables:** Variables with assigned labels having no quantitative value
Eg. Gender, Colours, Places, etc.

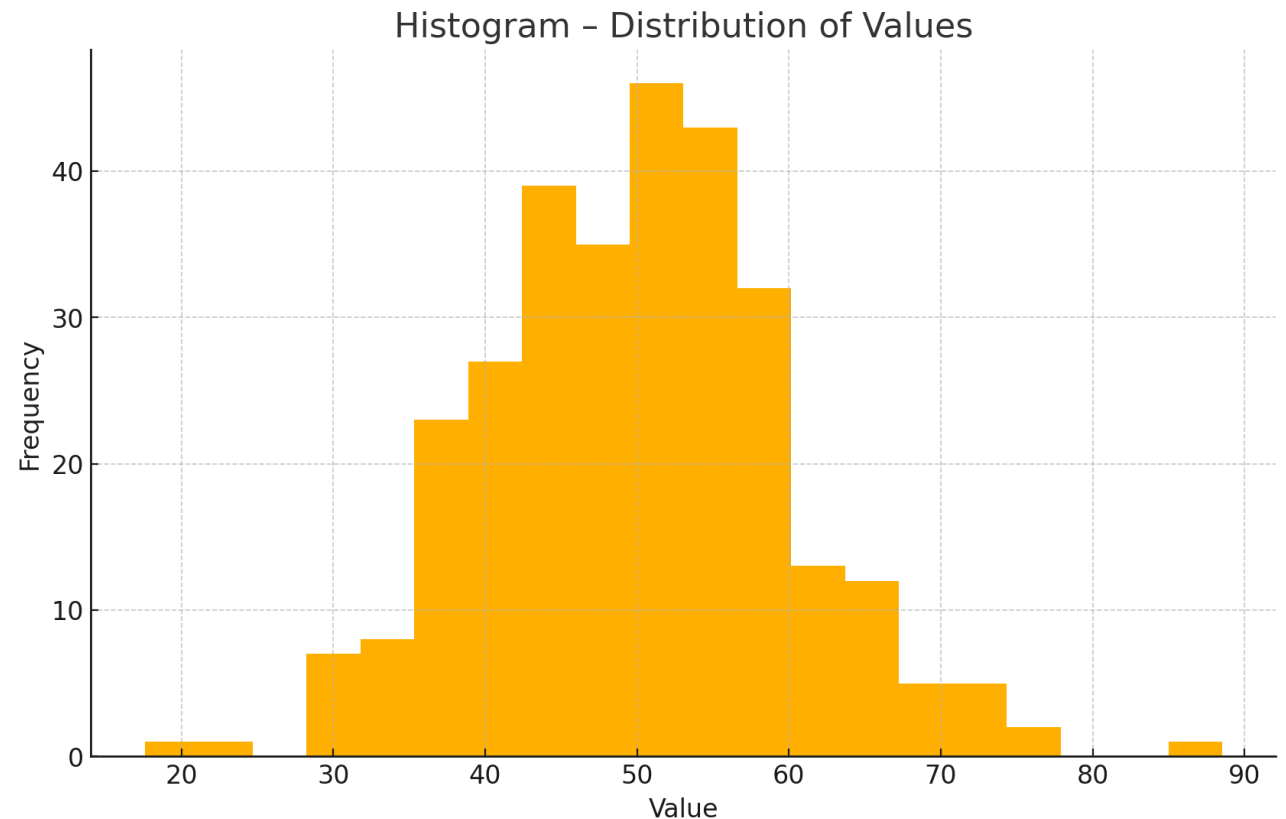
Types of Plots

Types of Plots – Based on Purpose

- Understanding distribution of a numerical variable (univariate):
 - Types of Plots:
 - Histogram
 - Box Plot
 - Violin Plot
 - Used to:
 - See the spread, shape and skewness
 - Understand range, centrality, and detect outliers

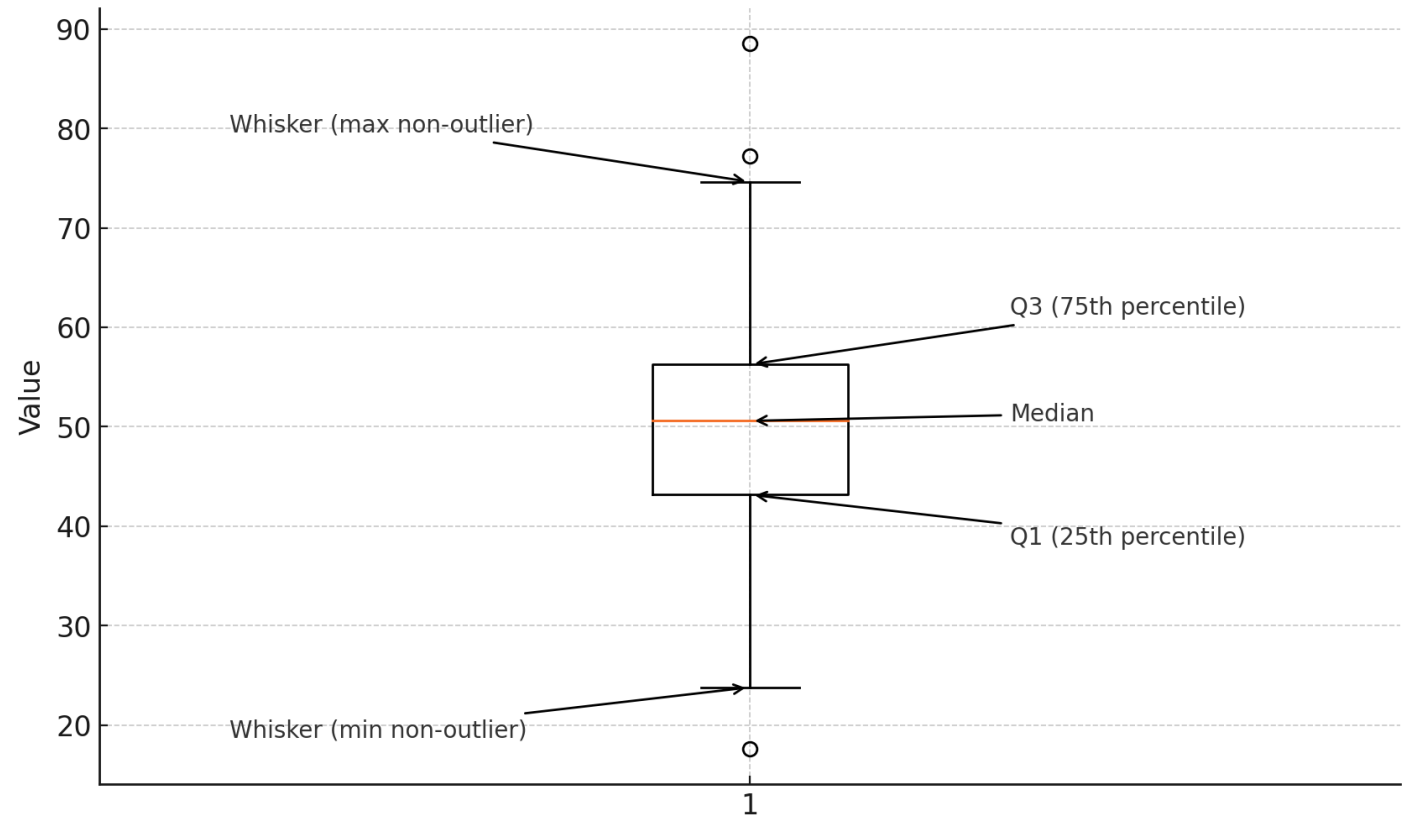
Histogram

- Plot of bars showing how many data points fall into numeric ranges called bins
- Also referred to as frequency plot
- Helps understanding the shape and frequency of values of a variable



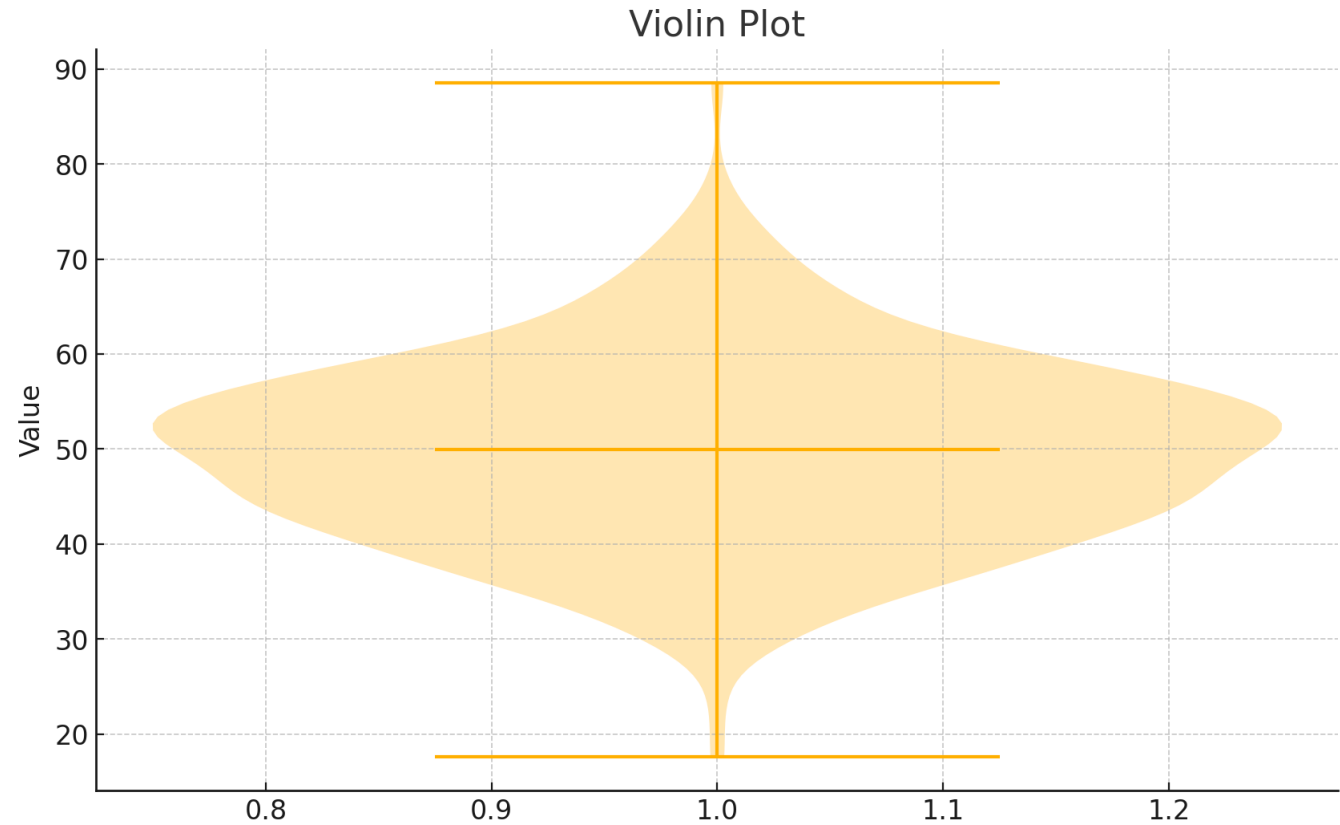
Box Plot

- A plot showing the spread and summary of a numeric variable using quartiles
- Used to compare statistical ranges and quartiles across variables
- Used to identify outliers



Violin Plot

- A plot showing both the spread and shape of a numerical variable
- Shape is a smooth curve estimation of the distribution symmetrically drawn around the box plot

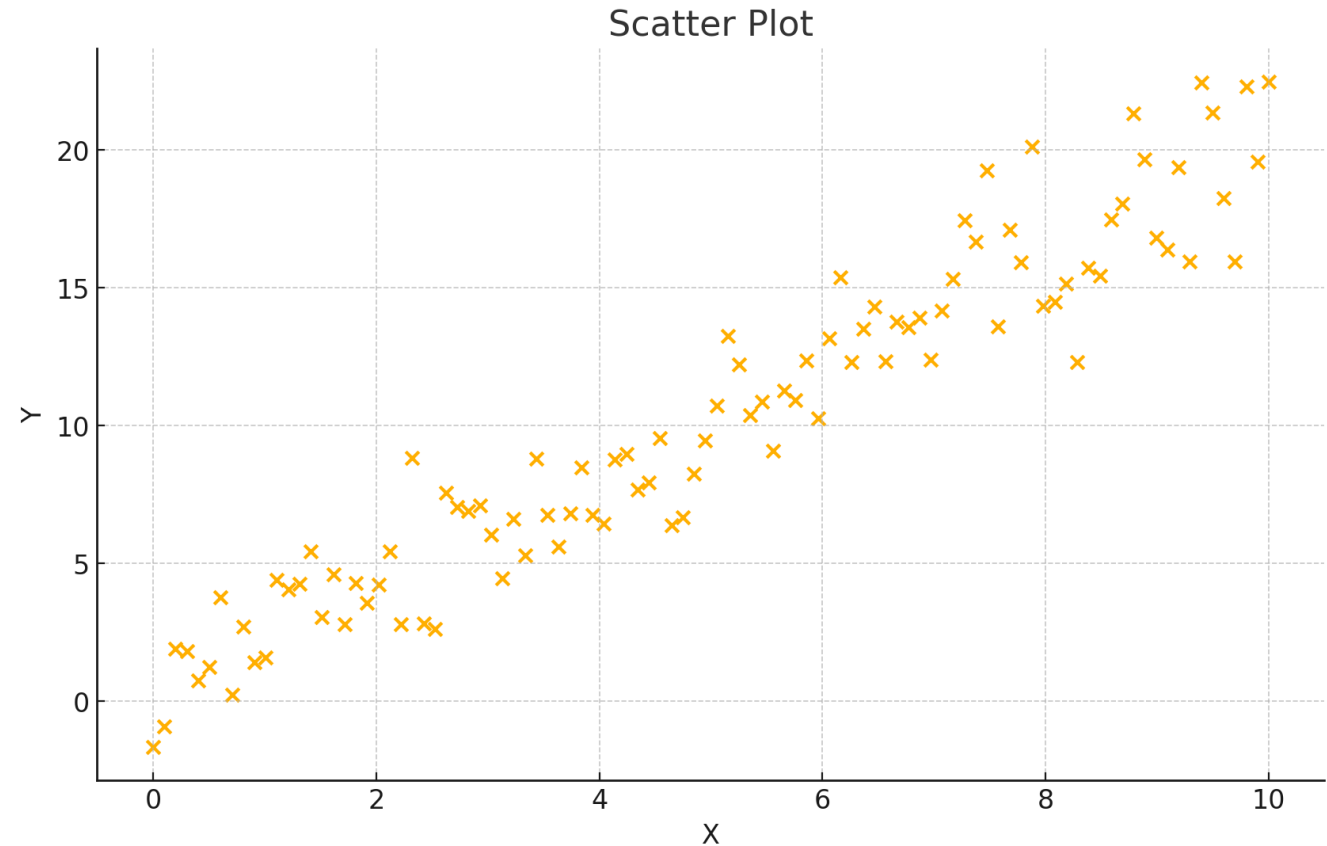


Types of Plots – Based on Purpose

- Understanding relation between two variables (multi-variate)
 - Types of Plots:
 - Scatter plot
 - Line Plot
 - Bubble Plot
 - Heatmap
 - Parallel coordinate plot
 - Used to:
 - To find correlations or patterns or trends
 - Observing changes over time

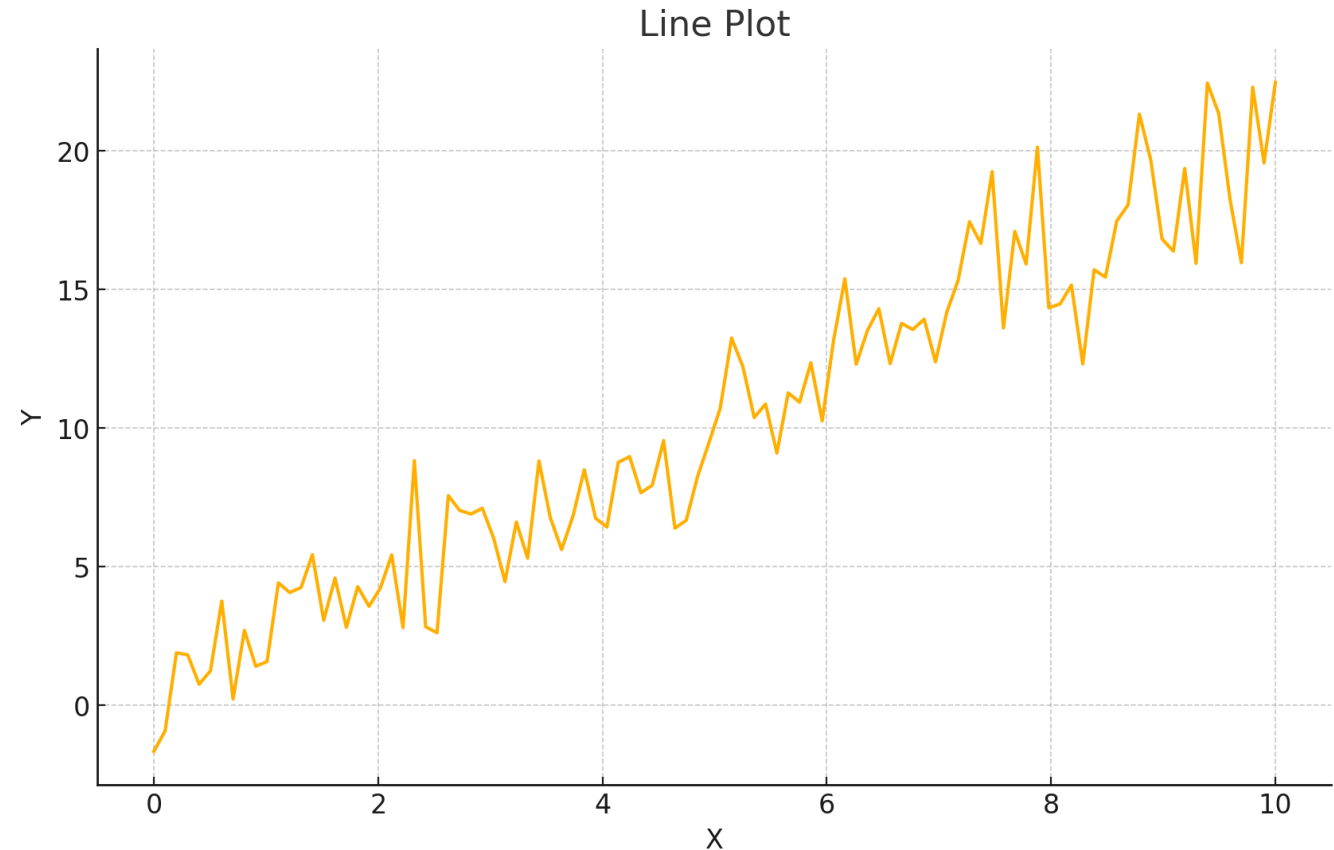
Scatter Plot

- Plot showing the relationship between two numerical variables
- Used for checking correlation, clusters, etc.



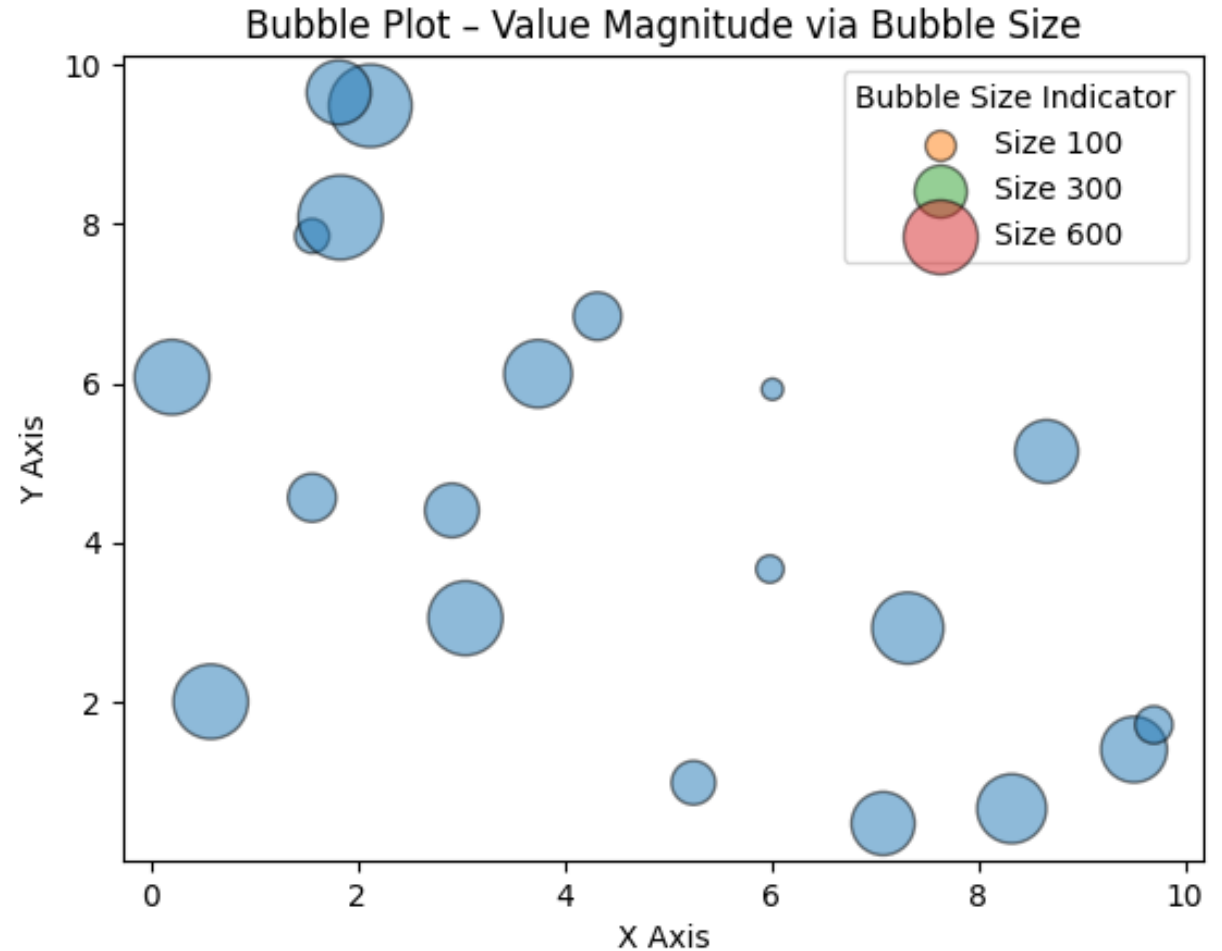
Line Plot

- Continuous line connecting data points in a defined order
- Ordering can be based on time or an ordered index
- Used to understand trends or patterns over time or ordered index



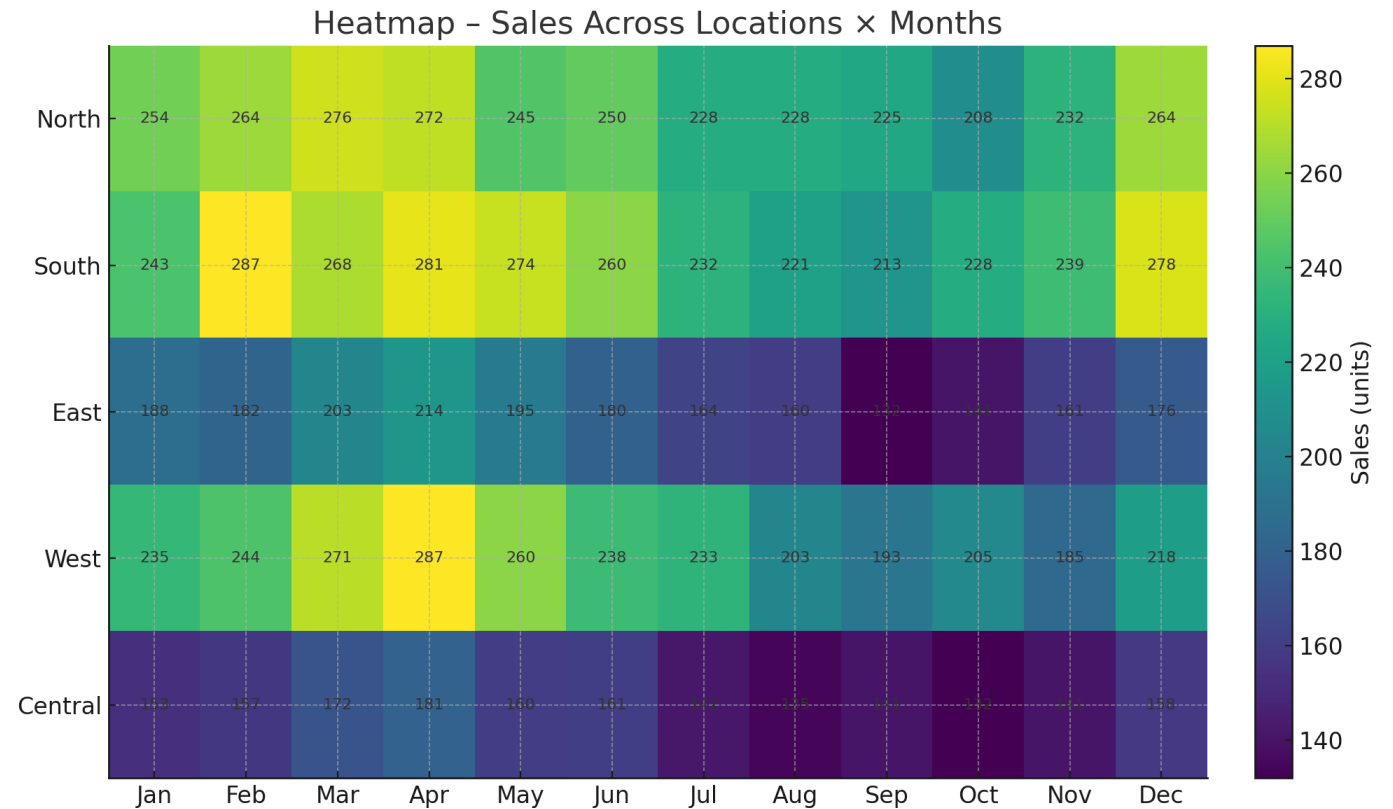
Bubble Plot

- Scatter plot where the dot size indicates a 3rd variable
- Further, bubble colour can also be used to represent a 4th variable (categorical)
- Eg:
 - x: Temperature of a city
 - y: Ice Cream Sales in the city
 - Bubble size: City population
 - Bubble colour: City Region



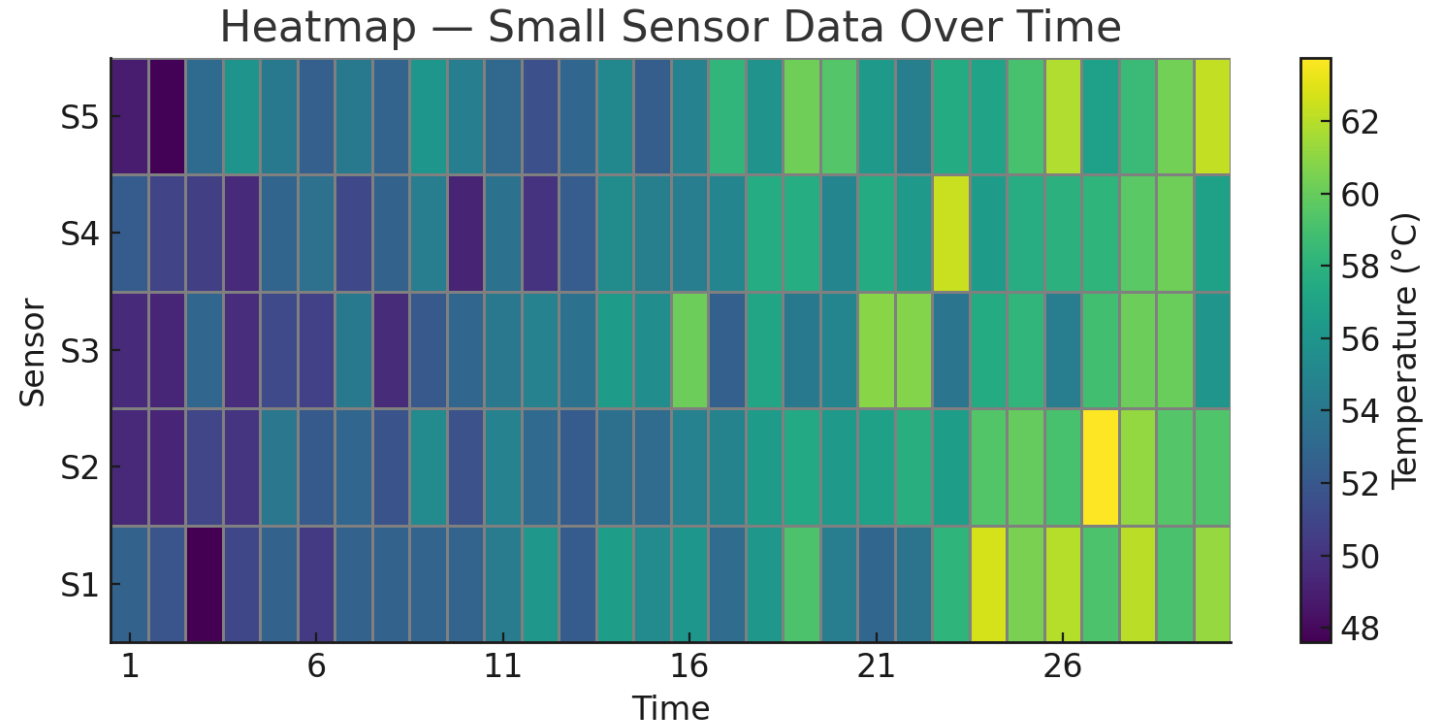
Heatmap

- Visualizes values in a matrix format using color intensity
- Can be used to visualize one numerical variable across two categoricals



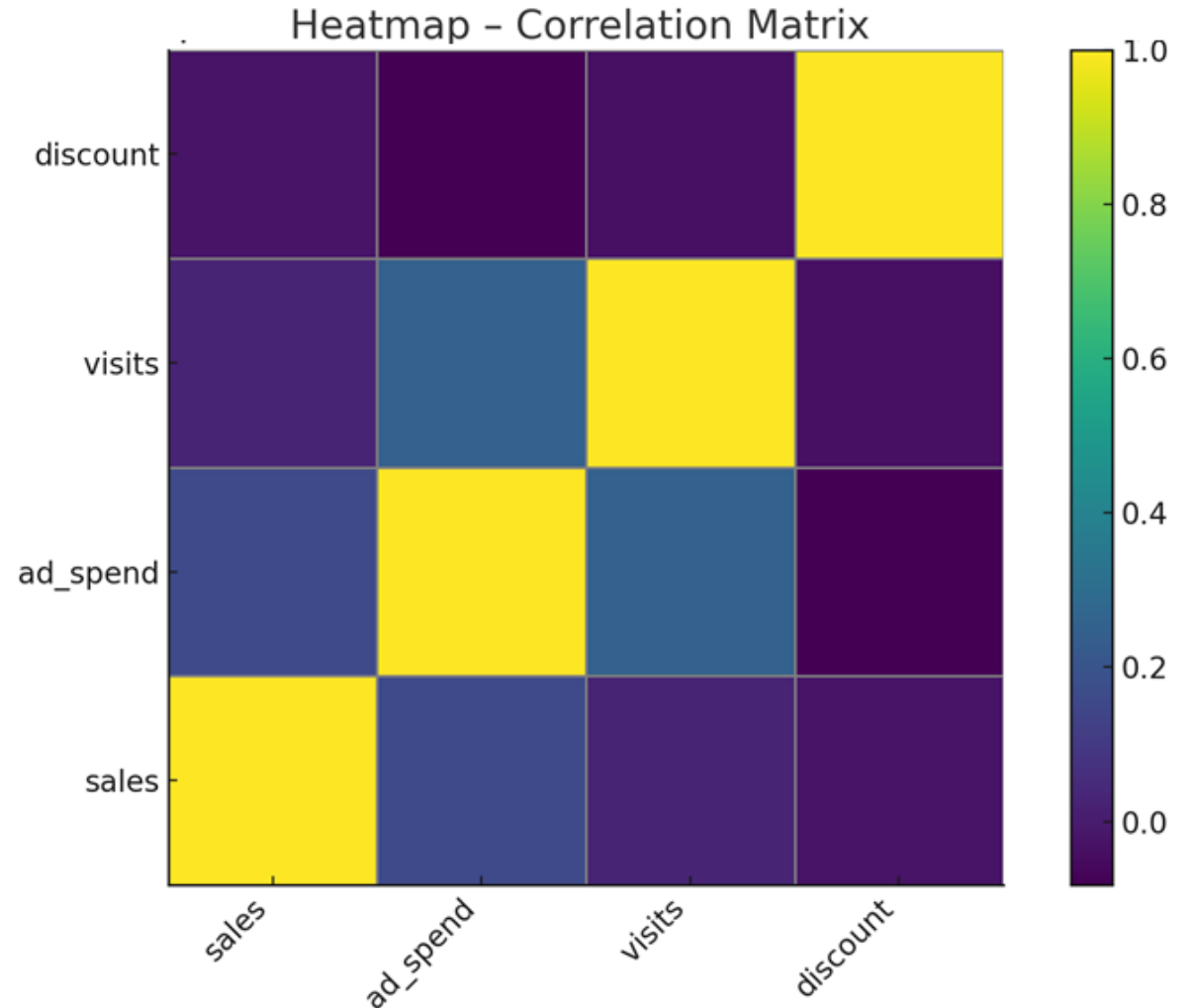
Heatmap

- Visualizes values in a matrix format using color intensity
- Can be used to visualize one numerical variable across two categoricals
- Can be used to visualize spacial or temporal patterns in process or sensor data



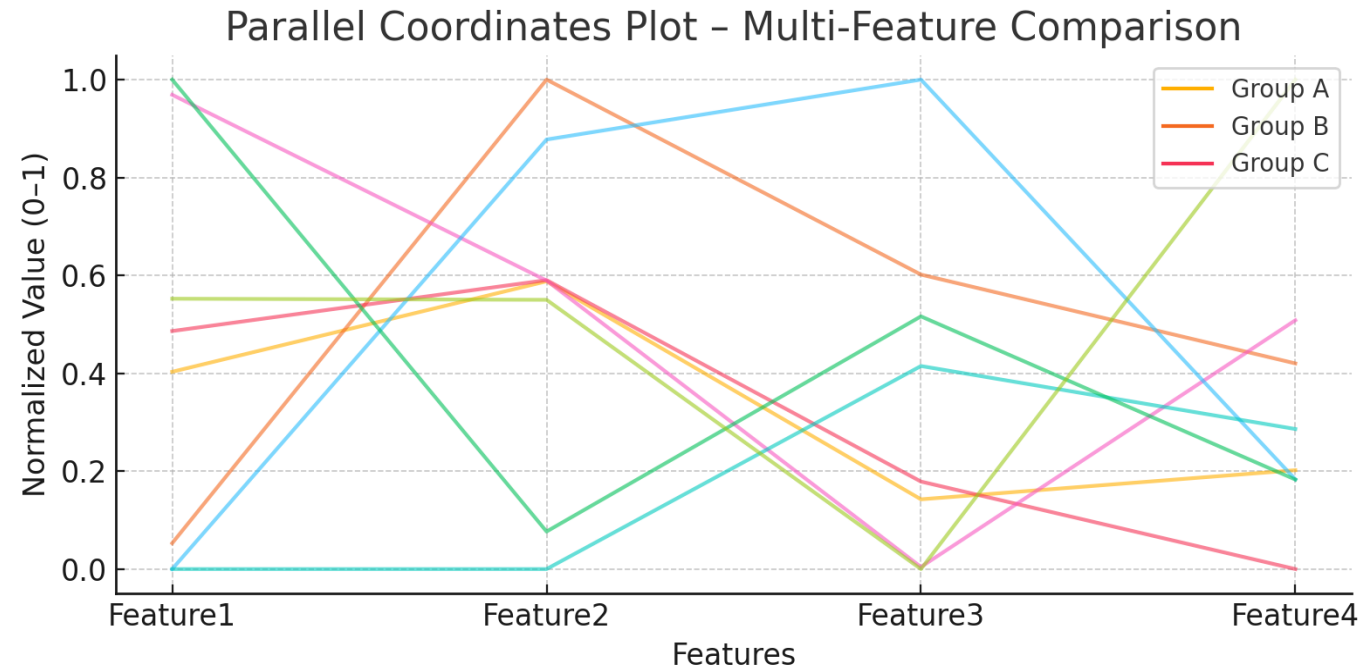
Heatmap

- Visualizes values in a matrix format using color intensity
- Can be used to visualize one numerical variable across two categoricals
- Can be used to visualize spacial or temporal patterns in process or sensor data
- When to use:
 - To understand relationships across multiple variables
 - To identify patterns in structured data (tables/matrices)
 - To identify high and low correlations



Parallel Coordinate Plot

- A visualization for comparing multiple numeric features across observations by drawing one polyline per observation, with one vertical axis per feature
- Colours of the polyline can be used to represent a categorical variable
- **Purpose**
 - Visualize patterns across many variables simultaneously
 - Identify clusters, outliers, and trends when data has multiple dimensions
 - Useful for multi-feature comparison where scatter plots aren't enough

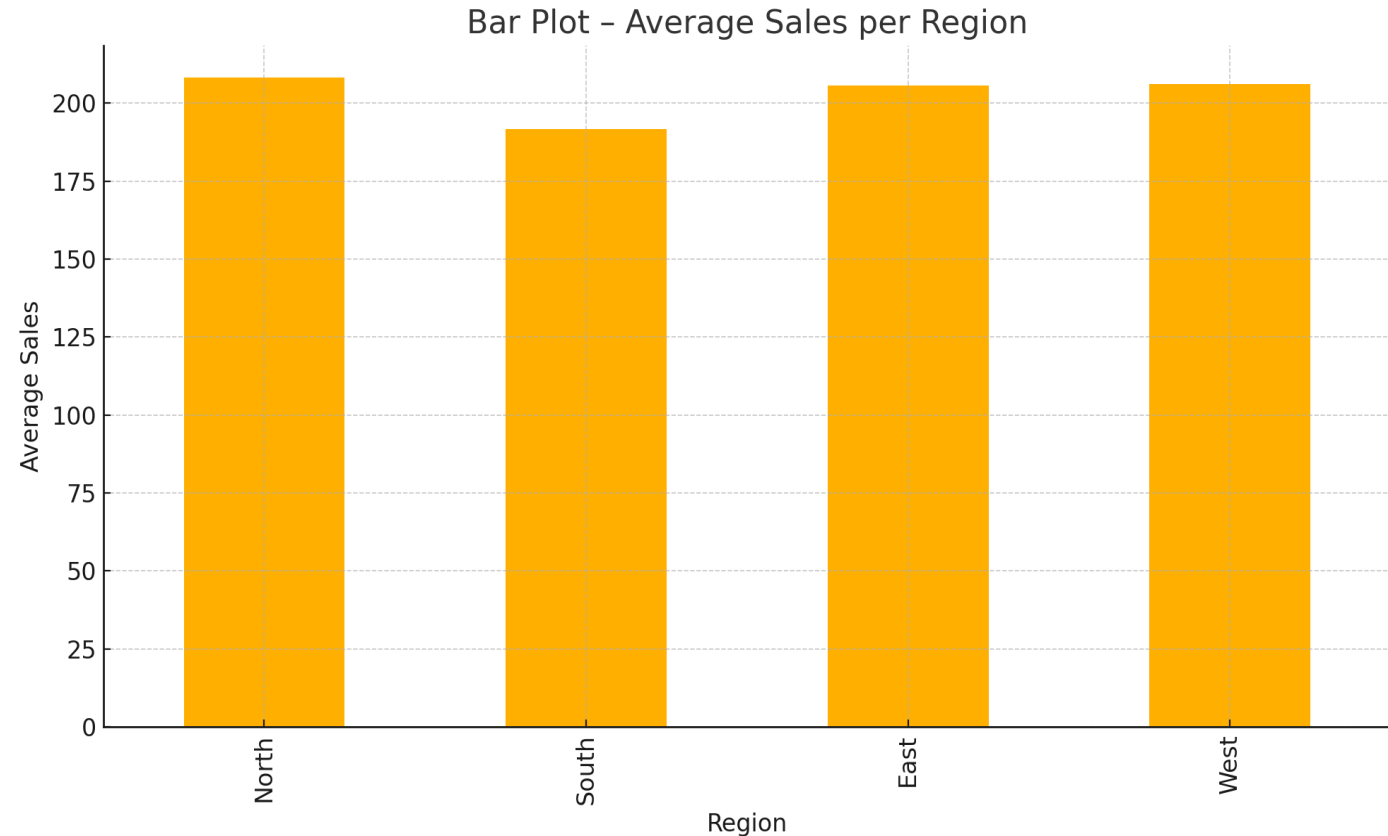


Types of Plots – Based on Purpose

- Comparing Categorical Variables and/or Summaries
 - Types of Plots:
 - Bar Plot
 - Count Plot
 - Pie Chart
 - Used to:
 - Understanding frequency or comparison across categories
 - Summarizing numeric data for categories

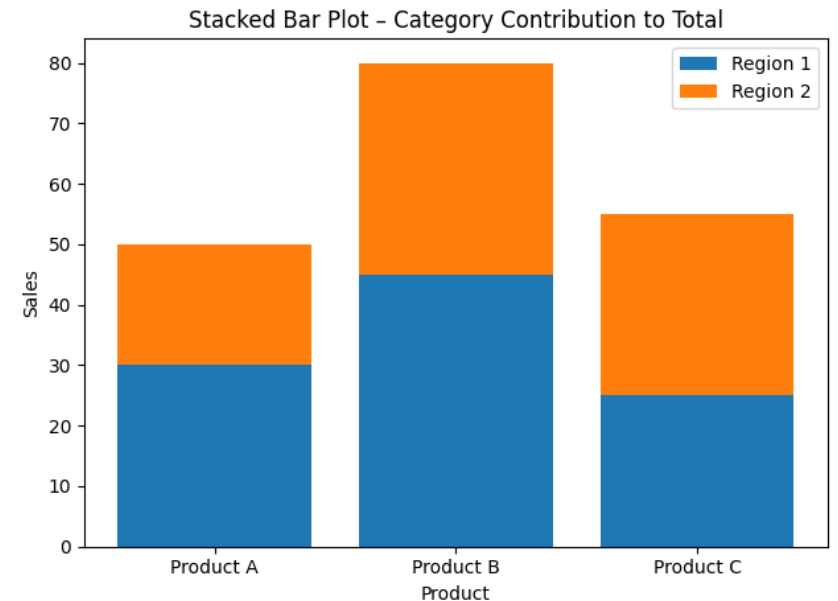
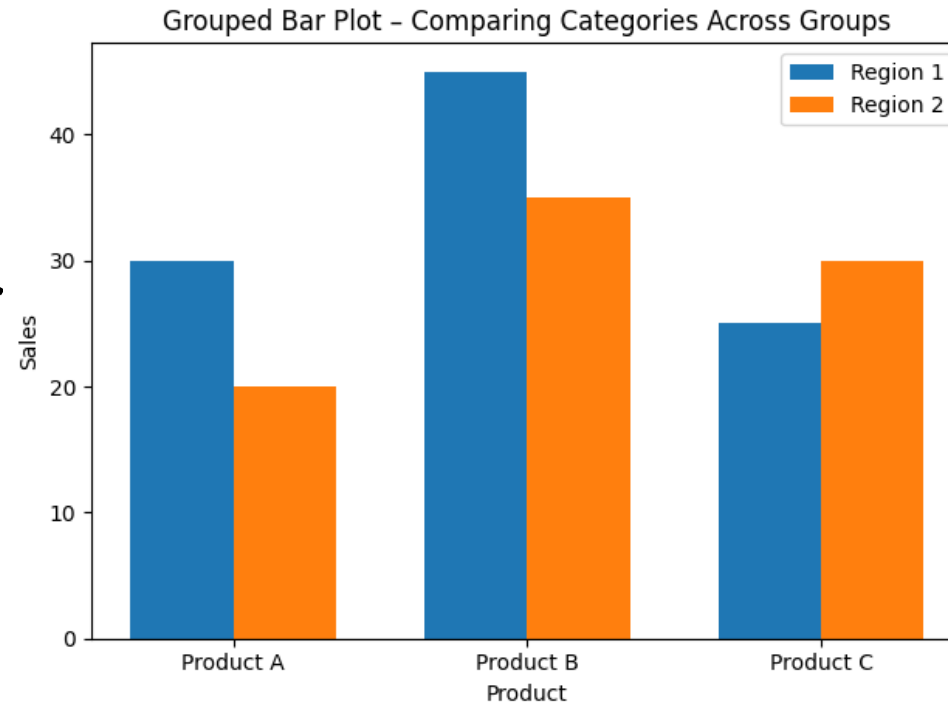
Bar Plot

- Represents values associated with categories using rectangular bars
- The height of each bar indicates the magnitude of the value (raw values, counts, or summarized statistics such as mean/total)
- Used to compare categories clearly and visually identify which category has higher/lower values



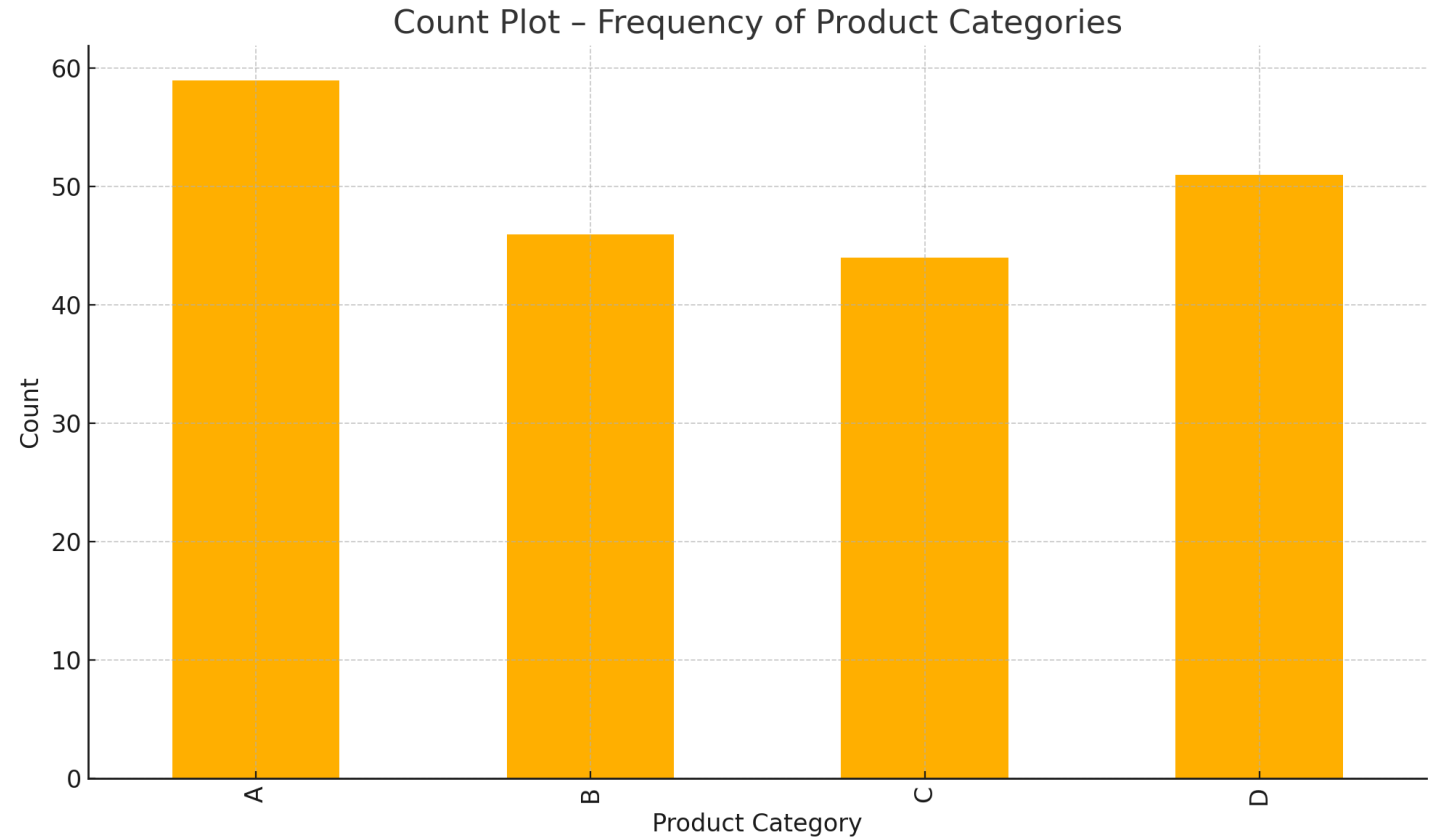
Bar Plot

- Represents values associated with categories using rectangular bars
- The height of each bar indicates the magnitude of the value (raw values, counts, or summarized statistics such as mean/total)
- Used to compare categories clearly and visually identify which category has higher/lower values
- Bar plots can be grouped and stacked as well



Count Plot

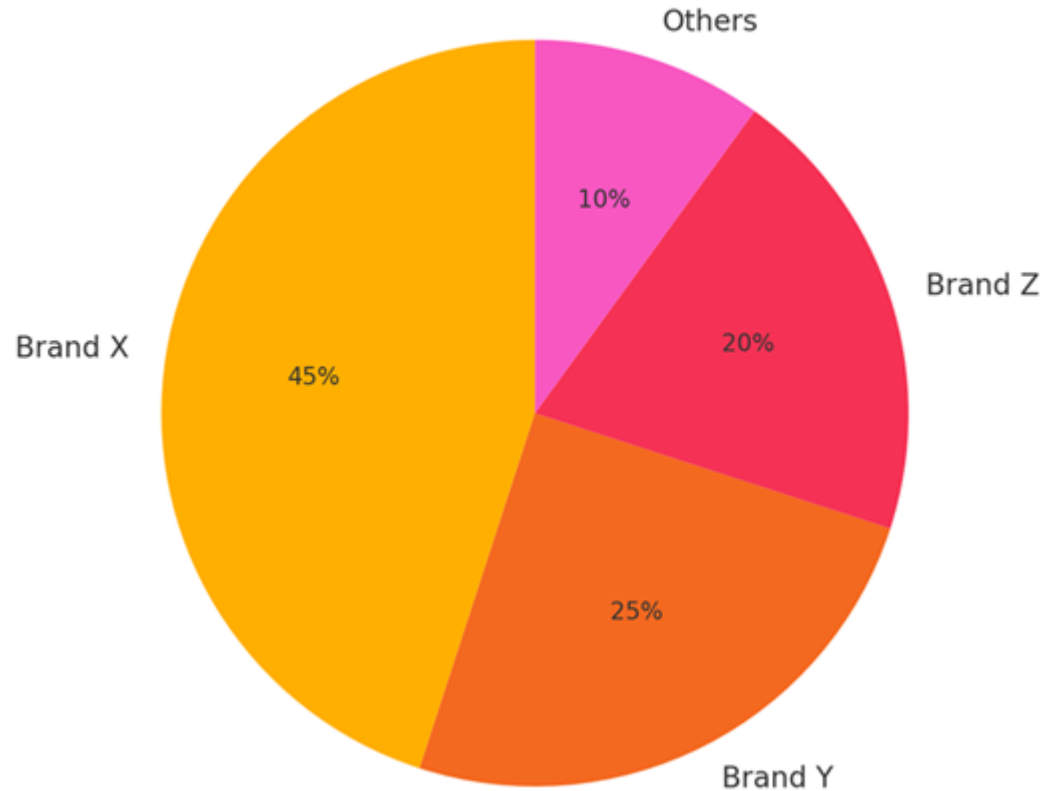
- A bar chart of frequencies showing how many records fall into each category
- Gives an idea of which category occurs the most or least



Pie Chart

- Circular chart partitioned into slices proportional to parts of a whole
- To compare different segments or categories
- **Tip:** Prefer bar charts when categories are many or close in size

Pie Chart – Market Share



Python Visualisation Ecosystem

Python Plotting Libraries

Matplotlib

- Foundation of Python visualization stack
- Full control over every visual element
- Supports scientific & publication-grade charts
- Highly customizable (fonts, axes, styles)

Seaborn

- Built on top of Matplotlib
- Statistical plotting library for quick EDA
- Beautiful default themes & color palettes
- Handles whole dataframes directly

Plotly

- Interactive charts (zoom, hover, filter)
- Web-ready visualizations
- Works with tables, maps, time series, 3D plots
- Integrates with Dash for dashboards/apps

Matplotlib Basics

- Works with lists, NumPy arrays, and pandas Series
- Gives full control over plot elements (axes, ticks, colors, grids, fonts)
- Key Concepts:
 - **figure**: Overall plotting canvas for which dimensions can be specified (sheet of paper)
 - Multiple objects can be placed on this canvas
 - Canvas can be saved
 - **axes**: Python object that contains and controls a plot placed within a figure (like boxes on paper)
 - **plt**: Convenient handle for matplotlib objects - keeps track of the current figure and current axes automatically

Seaborn Basics

- Built on top on matplotlib but makes the plots appear prettier
- Uses automatic themes, colour palettes and styling
- Can work directly with dataframes
- Provides high level functions for plotting wide range of statistical visualisations based on numerical as well as categorical variables
- Overall, it provides fast and clean statistical visualisations with minimal code

Plotly Basics

- Interactive by default: Hover tool tips, zoom/pan, select, etc.
- Visuals which can be rendered on browser and apps easily as it uses JavaScript engine
- Provides a high-level API for fast charts and low-level API for fine control
- Supports animations, subplots, 3D plots, maps, etc.
- Core Concepts:
 - **Figure:** Python objects that saves plot related details in the form of a nested dictionary and is converted to JSON for rendering
 - **plotly.express:** Module which has high level functions to create plots with one-line commands
 - **graph_objects:** Module which has low level functions to create fully customizable plots

Best Practices in Data Visualisation

- Know what is the intent and purpose of the plot – Avoid plotting without a purpose
- Choose the right type of plot – match it to the data and purpose
- Keep it simple and avoid unnecessary styling
- Add clear title and axis labels with units
- Use colour with purpose and don't overuse them
- Use consistent axis scales when comparing multiple charts